

Índice do Manual

1. Introdução Geral	9. ALERTA: Sem Logs Brutos nos `md`
2. O Conceito da Pasta `skills`	10. Passo a Passo de Uso no Dia a Dia
3. Arquivos de Regra vs. Operacionais	11. Exemplo Prático de Fluxo de Trabalho
4. Explicação de Cada Arquivo	12. Boas Práticas
5. O Guia Mestre: `00_START_HERE.md`	13. Erros Comuns
6. Arquivos de Contexto (01 a 06)	14. Benefícios para o Time
7. Arquivos Prontos vs. A Completar (IMPORTANTE)	15. Apêndice: Visão Resumida
8. Arquivos Operacionais (07 a 09)	

1. Introdução Geral

A inteligência artificial (ferramentas como Codex, ChatGPT, Cursor, Windsurf) revolucionou a velocidade em que escrevemos código. No entanto, ela introduziu um novo e crítico problema: **a amnésia de contexto e a entropia de código**.

Por que usar IA sem um contexto persistente gera bagunça?

- A IA não possui memória entre sessões. Ela não sabe o que foi decidido ontem ou por outro membro da equipe.
- Ela tende a reformatar códigos que já estavam funcionando por não entender o padrão arquitetural escolhido para aquele projeto.
- Gera inconsistências severas (ex: usar Tailwind em um componente e CSS puro em outro).
- Gera retrabalho ao recriar componentes que já existem na base de código.

A Solução: O Método `skills`

Este método resolve a perda de contexto criando um "cérebro central" para o projeto. Através de uma pasta com arquivos `skills` padronizados, garantimos que **agentes de IA e desenvolvedores humanos leiam a mesma fonte de verdade**. Isso melhora drasticamente a continuidade, a organização, a qualidade corporativa do código, a segurança e a produtividade.

2. O Conceito da pasta `skills`

A regra de adoção é extremamente simples e inegociável: **Na raiz de cada projeto de software, deve existir uma pasta chamada `skills` e unicamente `skills/`**.

Essa pasta atua como o *contexto permanente* e a *memória operacional* do sistema.

 **Para a Inteligência Artificial**

Funciona como as instruções de sistema globais. O agente é instruído a ler esta pasta para saber como programar, qual a stack tecnológica, o que foi feito ontem, e principalmente, o que *não* fazer.

 **Para os Desenvolvedores**

Funciona como a documentação técnica principal e diário de bordo. Elimina decisões contraditórias, acelera o onboarding de novos membros e garante continuidade caso um colega saia de férias.

3. Diferença entre Arquivos de Regra e Operacionais

Para manter a organização, os arquivos dentro de `skills/` são divididos em duas naturezas lógicas fundamentais:

Arquivos de Regra e Referência (00 a 06)

"Como o projeto deve ser conduzido."

São a fundação. Eles estabelecem as leis estáticas do projeto: a arquitetura escolhida, as regras de segurança, o design system. Eles **mudam pouco** após o início do projeto. A IA deve usar estes arquivos como barreiras de compliance para obedecer.

Arquivos Operacionais (07 a 09)

"O que aconteceu no projeto."

São a memória viva. Eles são um **diário de bordo** e **mudam constantemente**. Eles **registram o histórico de hoje**, as decisões tomadas nesta sprint e as pendências deixadas para amanhã. A IA e os humanos devem **atualizar** estes arquivos a cada nova implementação.

4. Explicação Individual de Cada Arquivo

O método `skills` exige uma estrutura fixa de 10 arquivos (numerados de 00 a 09). Entender o objetivo exato de cada um é o segredo para não gerar ruído no contexto da IA. Nas próximas seções (5 a 8), detalharemos o papel vital de cada arquivo e como o fluxo funciona.

5. `00_START_HERE.md` : O Guia Mestre

Este é o maestro da orquestra. **É obrigatório que ele seja o primeiro arquivo lido pelo agente de IA em qualquer interação.**

- Objetivo:** Organizar o comportamento do agente, definir sua "Persona" (ex: Tech Lead Sênior) e orquestrar a leitura dos demais arquivos.
- Quando usar:** Sempre que abrir o `Cursor/Windsurf` ou iniciar um novo chat (ex: "Leia o `skills/00_START_HERE.md` e *espere instruções*").
- Como ajuda:** Garante que a IA não invente fluxos de trabalho e saiba exatamente onde encontrar as regras de UI ou onde anotar o Changelog.

6. Arquivos de Contexto (Regras 01 a 06)

Estes arquivos definem os limites e padrões técnicos que a IA não pode violar.

`01_PRODUCT_SCOPE.md`

Objetivo: Define o que é o produto, suas personas e funcionalidades core.

Evita: Que a IA sofra de "Scope Creep" criando telas complexas para features não planejadas.

`02_ARCHITECTURE.md`

Objetivo: Define a stack técnica exata, padrões de projeto (MVC, Clean) e bibliotecas.

Evita: Que a IA instale pacotes aleatórios (ex: instalar `axios` quando a regra diz para usar o nativo `fetch`).

`03_SECURITY.md`

Objetivo: Regras estritas de segurança (tratamento de CORS, sanitização de inputs, gestão de JWT).

`04_QA.md`

Objetivo: Critérios de aceitação, padrões de testes unitários e e2e (Jest, Cypress).

`05_UI_UX_RULES.md`

Objetivo: As regras visuais. Paleta de cores, tipografia, uso de Tailwind, regras de responsividade estritas.

Evita: Interfaces feias, inconsistentes ou componentes não responsivos.

`06_CODE_RULES.md`

Objetivo: Regras de linting e sintaxe humana. Ex: "Sempre use early returns", "Nomeie em camelCase".

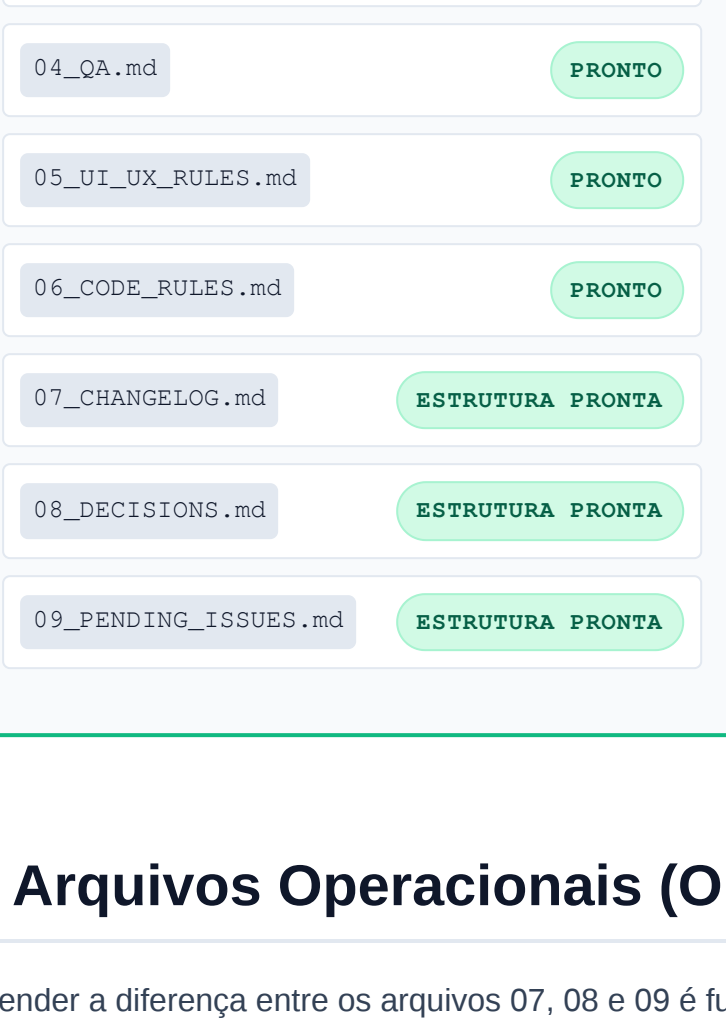
7. Arquivos Prontos vs. A Completar (CRÍTICO)

O maior ganho de velocidade deste método é que **você não precisa reinventar a roda a cada projeto**. A esmagadora maioria da estrutura já está formatada.

Abaixo, a divisão visual do que já vem pronto no template e o que exige esforço da sua equipe ao iniciar um novo projeto.

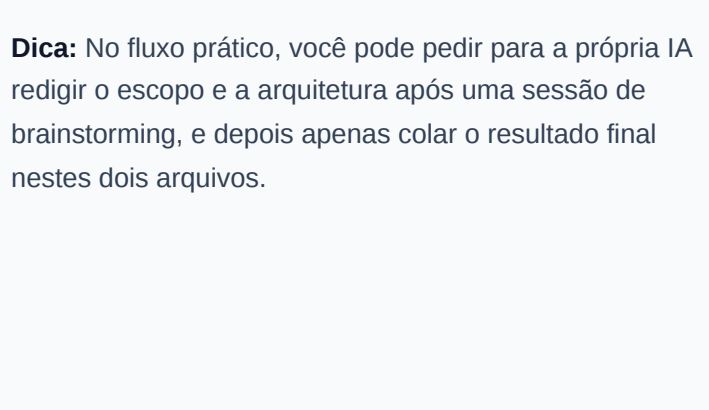
Já Prontos (Padrão do Time)

Estes arquivos podem ser usados como estão. Eles contêm as melhores práticas de engenharia da equipe e só precisam de ajustes se algo muito atípico ocorrer no projeto.



Precisam ser Completados

Estes dois arquivos possuem a estrutura base pronta, mas **precisam ser preenchidos** com as informações reais e específicas do novo projeto logo no dia 1.



Dica: No fluxo prático, você pode pedir para a própria IA redigir o escopo e a arquitetura após uma sessão de brainstorming, e depois apenas colar o resultado final nestes dois arquivos.

8. Arquivos Operacionais (O Diário de Bordo)

Entender a diferença entre os arquivos 07, 08 e 09 é fundamental para não misturar contexto.

Arquivo	O que registra	Exemplo Prático
<code>07_CHANGELOG.md</code>	O que foi feito. Histórico consolidado (features prontas).	"Implementado sistema de login com Google."
<code>08_DECISIONS.md</code>	O que foi decidido e por quê. Justificativas para não voltar atrás.	"Decidimos usar Zustand em vez de Redux porque o projeto é pequeno e requer menos boilerplate."
<code>09_PENDING_ISSUES.md</code>	O que falta fazer / Débitos. Bugs não resolvidos, limitações.	"O botão de submit não desabilita em conexões lentas (aguardando refatoração)."

9. ALERTA CRÍTICO: Não use `md` para Logs Brutos

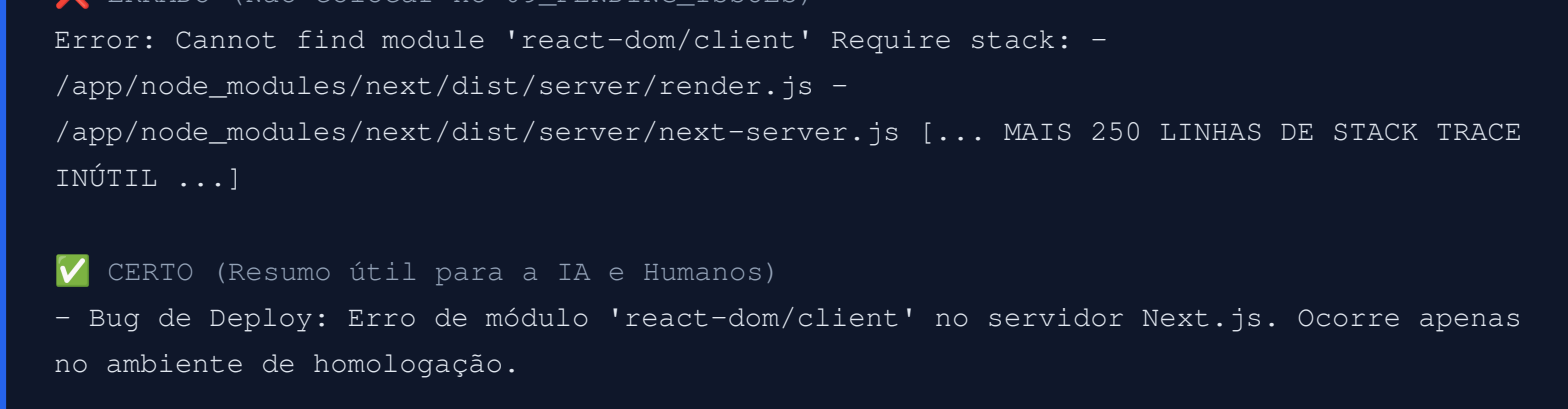
 **Arquivos `.md` guardam CONTEXTO HUMANO, não lixo digital!**

É estritamente proibido colar logs completos de console, stack traces gigantescos, dumps infinitos de terminal ou logs de build do CI/CD dentro da pasta `skills/`.

Por que isso destrói o projeto?

- A janela de contexto (memória) da IA é limitada (Token Limit).
- Ao encher o arquivo com milhares de linhas de lixo, a IA é forçada a "esquecer" as regras críticas de arquitetura para acomodar o log.
- O arquivo fica ilegível para desenvolvedores humanos.

Como registrar erros corretamente:



 **CERTO** (Resumo útil para a IA e Humanos)
- Bug de Deploy: Erro de módulo 'react-dom/client' no servidor Next.js. Ocorre apenas no ambiente de homologação.

10. Passo a Passo de Uso no Dia a Dia

Siga este roteiro rigoroso para extrair 100% de eficiência da IA.

- 1. Criação:** Ao iniciar um repositório, crie a pasta `skills/` na raiz.
- 2. Importação:** Copie todos os arquivos padrão (00 a 09) do repositório template da empresa.
- 3. Preenchimento (Setup):** Mantenha os arquivos `00` e `03` intactos. Preencha rigorosamente o `01_PRODUCT_SCOPE.md` e o `02_ARCHITECTURE.md` com as realidades do novo projeto.
- 4. Acionamento:** Abra sua IDE com IA e digite o prompt de ouro:
"Leia a pasta `skills/`, começando obrigatoriamente pelo `00_START_HERE.md`. Confirme que entendeu as regras antes de começarmos."
- 5. Desenvolvimento:** Peça à IA para implementar as features. Ela obedecerá às regras automaticamente.
- 6. Encerramento:** Ao fim de uma sessão longa, instrua a IA:
"Atualize o `07_CHANGELOG.md` com o que fizemos hoje. Se deixamos algo pendente, anote no `09_PENDING_ISSUES.md`."

11. Exemplo Prático de Fluxo de Trabalho

Cenário: Equipe iniciando um novo MVP de e-commerce.

- A equipe cria o repositório e copia a pasta `skills`.
- O Tech Lead abre o `02_ARCHITECTURE.md`, que estava com uma estrutura base, e completa: "Usaremos `Next.js 14 App Router`, `Supabase` para `Auth`, e `Stripe` para pagamentos."
- O dev júnior abre o `Cursor`, pede para a IA ler o `00_START_HERE.md` e solicita: "Crie o componente de *carrito de compras*."
- A IA lê as regras. Sem fazer perguntas desnecessárias, ela usa componentes do Tailwind (regra 05), cria funções server-side (regra 02) e aplica sanitização (regra 03).
- Durante o processo, a equipe percebe que o Stripe não aceita Pix na versão atual. A IA sugere uma biblioteca alternativa temporária.
- O Dev manda: "Registre essa limitação do Stripe no `08_DECISIONS.md` para que o time saiba por que usamos essa lib secundária."
- Ao fechar o PR, a IA atualiza o `07_CHANGELOG.md` com a entrega do carrinho.

12. Boas Práticas (Golden Rules)

- Aproveite o Padrão:** Nunca recrie arquivos do zero se eles já vieram no template. Complete corretamente apenas escopo e arquitetura.
- Manutenção como Código:** A pasta `skills/` deve passar por Code Review (PR) igual a qualquer arquivo `.js` ou `.py`.
- Seja Consistente:** Escreva de forma clara e direta (bullet points são excelentes). A IA não precisa de prosas literárias.
- Sincronia:** Se a arquitetura mudar na vida real, altere o arquivo `02_ARCHITECTURE.md` no mesmo commit.
- Auto-Revisão da IA:** Periodicamente, peça à IA: "Analise nosso código atual e veja se os arquivos 07, 08 e 09 na pasta `skills` estão desatualizados."

13. Erros Comuns que Quebram o Método

- Achar que precisa recriar tudo em cada projeto:** Ignorar os arquivos já prontos e perder tempo recriando regras de lint ou qualidade que são padrão da empresa.
- Deixar a Arquitetura (02) vaga:** Escrever "Usamos React" em vez de "Usamos React 18, Vite, Zustand para estado local e React Query para cache server-side".
- Polição de Logs:** Já avisamos, mas vale repetir: colar `npm ERR!` brutos dentro do `09_PENDING_ISSUES.md`.
- Misturar Histórico com Decisão:** Colocar no Changelog "Mudamos para AWS porque a Vercel estava cara". A mudança vai pro Changelog. O motivo ("estava cara") vai pro Decisions.
- Backlog Disfarçado:** Usar o `09_PENDING_ISSUES.md` como se fosse o Jira. Ele serve para contexto técnico resumo (débitos, limitações de lib), não para planejar os próximos 6 meses de features.

14. O ROI: Benefícios Imediatos para o Time

Cenário Comum (Sem Skills)	Resultado (Com Método Skills)
Setup de projeto lento e burocrático.	Início rápido, pois 80% das regras já vêm prontas nos arquivos padrão.
A IA "esquece" o que fez ontem e refatora o componente errado.	O <code>00_START_HERE.md</code> garante que ela leia o estado atual antes de agir.
Membro da equipe sai de férias e o conhecimento "vai com ele".	A memória fica no <code>08_DECISIONS</code> e <code>09_PENDING_ISSUES</code> . Qualquer IA ou humano continua de onde parou.
Cada dev usa a IA para gerar interfaces com um padrão visual diferente.	Padronização absoluta forçada pelo arquivo <code>05_UI_UX_RULES</code> .

15. Apêndice: Resumo Executivo dos Arquivos

Use esta tabela como consulta rápida.

`00_START_HERE.md`

PRONTO

Guia principal de orientação, persona da IA e mapa de orquestração.

`01_PRODUCT_SCOPE.md`

COMPLETAR

Descrição do produto, persona do usuário e delimitação funcional (evita scope creep).

`02_ARCHITECTURE.md`

COMPLETAR

Stack tecnológica exata, infraestrutura, padrões de comunicação e diretórios.

`03_SECURITY.md`

PRONTO

Leis rigorosas de segurança, proteção de dados e manuseio de credenciais.

`04_QA.md`

PRONTO

Padrões de testes, linting, cobertura e processos de homologação.

`05_UI_UX_RULES.md`

PRONTO

Design System, cores hexadecimais, espaçamentos e regras de responsividade.

`06_CODE_RULES.md`

PRONTO

Boas práticas de código, nomenclatura, clean code e tratamento de erros.

`07_CHANGELOG.md`

ESTRUTURA PRONTA

Histórico contínuo do que foi efetivamente desenvolvido no projeto (Atualizado constantemente).

`08_DECISIONS.md`

ESTRUTURA PRONTA

Registro arquitetural (ADR). Qual tecnologia/caminho escolhemos e o motivo. (Atualizado por demanda).

`09_PENDING_ISSUES.md`

ESTRUTURA PRONTA

Débitos técnicos de curto prazo, limitações conhecidas e bugs em aberto. (Atualizado constantemente).